

# CS 378 GFML: First Phase; ScatterNeRF

Brian Jiang

September 2025

## 1 Problem Setup and Prior Work Survey

### 1.1 Problem Formulation

Given some scene  $S$ , we have a set of about 150-300 foggy images  $X_S^{(i)}$  of the scene. ScatterNeRF learns a neural network that jointly predicts the object and scatter color and density.

We train a standard NeRF model that predicts the color and density of objects.

$$\begin{aligned}\text{NeRF}_s : \mathbb{R}^5 &\rightarrow \mathbb{R}^4 \\ \text{NeRF}_s(\mathbf{x}, \mathbf{d}) &= (\mathbf{c}_s, \sigma_s)\end{aligned}$$

Where  $\mathbf{x}$  is position vector for some object,  $\mathbf{d} = (\phi, \theta)$ , which describes the direction at which a ray crosses both  $\mathbf{x}$  and some camera pixel. A ray vector is parameterized by  $t$  as  $\mathbf{r} = \mathbf{x} + t\hat{\mathbf{d}}$ , where  $\mathbf{c}_c$  is the RGB color encoding associated with the opaque object, and  $\mathbf{c}_p$  is the RGB color encoding associated with the environment/scattering (i.e. haze, fog, etc.)

### 1.2 Prior Work Survey

1. NeRF Models: Framework is the same as shown above: given an object's location and camera ray vector, predict the object density and RGB encoding.
2. Mip-NeRF 360: Broadens possible scenes that make sense for NeRF models by fixing maximum depth distance on  $\mathbf{d}$ . Allows for scene reconstruction/generation of unbounded scenes.
3. ScatterNeRF: follows the framework above. Does this by introducing an additional MLP that it meant to capture the scattering properties. Unlocks both original scene reconstruction/generation and dehazed reconstruction.

## 2 Gap Analysis and Proposed Approach

### 2.1 Identify Gaps

1. Pervasive Gaps (weaknesses inherent to the problem)
  - (a) Lack of Good Training Data - difficult to get spherical coordinates of objects (often a large set of spherical coordinates due to object structure)
  - (b) Limitations of MLPs - NeRF models are generally implemented using a series of MLPs. MLP computations are expensive and difficult to interpret. MLPs are also prone to overfitting, so small perturbations in the input (small changes in  $\mathbf{x}, \mathbf{d}$  may lead to significantly different outputs, even though the vast majority of scenes are smooth.
2. Rejected Approaches: Synthetic Paired Data Generation - discussed in previous research to not generalize well to the real world.

### 2.2 Proposed Approach - Transformer Architecture

Main hope: Transformers can process multiple  $(\mathbf{x}, \mathbf{d})$  in parallel and learn the covariance between them. Hopefully this allows transformers to capture better locality (i.e., points close to each other in 3D space should in theory have similar outputs - RGB encoding and density).

## 3 Experimental Plans

### 3.1 Data and Evaluation:

Use the datasets and evaluation metrics provided in ScatterNeRF, so we can compare performance across both models: LPIPS, PSNR, SSIM (learned perceptual image patch similarity, peak signal-to-noise ratio, and structure similarity index measure).

### 3.2 Training/Testing/Validation

1. Training Procedure: 5-Fold Cross Validation on Datasets, with 64-16-20 split for train/validation/testing.
2. Hyperparameter Tuning: We will make an assumption about the size of the attention matrix (follow similar guidelines for sparse GPs to battle intractability problem). Optimizer: Adam, Learning Rate:  $1e-4$  (might tune this - trying different values, schedulers, etc.) Since Transformers are learning kernels, we don't have to do hyperparameter tuning on a preassumed kernel